COLLEGE OF CREATIVE INNOVATION

Master of Software Engineering — Level 8

MSE802 Quantum Computing

Assessment 2 — Quantum Project (60%)

Task 3 Report: Investigating a Quantum Code

Quantum Tic-Tac-Toe: Code Completion, Gameplay, Circuit Generation and Quantum Gates

GROUP PROJECT

Student 1	Anish Shrestha — Student ID 270889991
Student 2	Sandhya Tiwari — Student ID 270918511
Lecturer	Dr Helena Bahrami
Course	MSE802 Quantum Computing (15 credits)
Date	4 November 2026
GitHub repository	github.com/anishnz/quantumcomputing_YB_Anish
Referencing style	APA 7th edition

Auckland, New Zealand

Declaration of Originality

We, the members of this group named below, declare that this report and the accompanying code are our own group work, completed for MSE802 Assessment 2. All sources of information, code and ideas from others, including the lecture notes and the supplied lab notebooks, have been acknowledged and referenced in APA 7th style. This work has not been submitted for any other course. We understand Yoobee College's policies on plagiarism and collusion as described in the Student Handbook.

We confirm that the results reported here come from the notebooks submitted with this report. Results produced on a local simulator because the Quokka device was unavailable are clearly labelled as such and are not presented as device measurements.

Use of digital tools: Python libraries (Cirq, Qiskit, NumPy, Matplotlib) were used for all computation. A generative AI assistant was used to support code drafting, debugging and language editing; all code was run, checked and understood by the group, and the analysis and conclusions are our own. [Edit this sentence to match Yoobee's AI-use policy and your actual use.]

Name	Student ID
Anish Shrestha	270889991
Sandhya Tiwari	270918511

Project Links and How to Run Online

Every result in this document can be reproduced online, with nothing to install. Copy a link below and paste it into the address bar of a web browser.

GitHub repository (all code, results and reports):
https://github.com/anishnz/quantumcomputing_YB_Anish

Google Colab – run ALL tasks (recommended):
https://colab.research.google.com/github/anishnz/quantumcomputing_YB_Anish/blob/main/Assignment_2_Quantum/Colab_Run_All_Tasks.ipynb

Google Colab – Task 1 (Bell state, Cirq):
https://colab.research.google.com/github/anishnz/quantumcomputing_YB_Anish/blob/main/Assignment_2_Quantum/Task_1_Bell_State/Task1_Bell_State_Cirq_Quokka.ipynb

Google Colab – Task 2 (Qiskit circuit + QASM):
https://colab.research.google.com/github/anishnz/quantumcomputing_YB_Anish/blob/main/Assignment_2_Quantum/Task_2_Qiskit_QASM/Task2_Qiskit_Circuits_and_QASM.ipynb

Google Colab – Task 3 (quantum tic-tac-toe):
https://colab.research.google.com/github/anishnz/quantumcomputing_YB_Anish/blob/main/Assignment_2_Quantum/Task_3_Quantum_TicTacToe/Task3_Quantum_TicTacToe_Completed.ipynb

Google Colab – Task 4 (quantum machine learning):
https://colab.research.google.com/github/anishnz/quantumcomputing_YB_Anish/blob/main/Assignment_2_Quantum/Task_4_Quantum_ML/Task4_Quantum_ML_Modified.ipynb

Download the whole project as a ZIP:
https://github.com/anishnz/quantumcomputing_YB_Anish/archive/refs/heads/main.zip

How to run the project online in Google Colab (step by step)

1. Copy the link "Google Colab – run ALL tasks" above and paste it into the address bar of Chrome, Edge or Safari, then press Enter.
2. Sign in with any Google account if Colab asks you to (Colab is free).
3. Click Runtime → Run all. If Colab shows "Warning: This notebook was not authored by Google", click Run anyway.
4. Wait about 15–20 minutes. The notebook downloads the GitHub repository, installs Cirq and Qiskit, checks which Quokka devices are online, runs Tasks 1–4 with 1000 shots (Task 4 sends 900 circuits to Quokka), shows every table and chart, runs the laptop-vs-Quokka comparison and the alternative code.
5. Read the results under each step. Every result says where it ran: QUOKKA DEVICE (with the device name, e.g. quokka2) or LOCAL SIMULATOR if no Quokka device is online.
6. To keep your own copy, click File → Save a copy in Drive. Step 7 of the notebook can also download all outputs as a ZIP.
7. To run only one task, open that task's Colab link above, then click Runtime → Run all. Its first code cell downloads the project automatically.
8. To run on your own computer instead: download the ZIP link above, unzip it, open the Assignment_2_Quantum folder and double-click run_project.bat (Windows) or run_project.command (macOS). Python 3.12 or newer is needed.

Table of Contents

Declaration of Originality..... 1
Project Links and How to Run Online..... 2
Table of Contents..... 3
1. Task 3 – Investigating a Quantum Code: Quantum Tic-Tac-Toe..... 4
1.1 What is the purpose of the program?..... 4
1.2 How is the code organised?..... 4
1.3 What was missing, and what did we complete or fix?..... 4
1.4 How does a move become a gate? (Circuit generation)..... 5
1.5 How does a rotation move change the odds?..... 5
1.6 What happened when we played? (Four recorded games)..... 6
1.7 What do the games show? (Analysis)..... 7
1.8 Did we also play the game by clicking the buttons?..... 8
1.9 Can the game circuit run on Quokka?..... 9
1.10 Which quantum gates does the game use? (Gate section)..... 9
1.11 How do these gates fit the wider theory of quantum logic gates?..... 10
1.12 What are the strengths and limits of the program?..... 10
References..... 10

(If page numbers are missing, right-click the table and choose Update Field.)

> This is the separate Task 3 report requested by the brief. Its content is identical to Section 5 of the group report Assignment2.docx, which also contains the introduction, the other tasks and the full discussion.

1. Task 3 – Investigating a Quantum Code: Quantum Tic-Tac-Toe

> Task question: Complete the provided code by filling in missing instructions or comments; play multiple games and analyse the generated code; as a quantum software engineer, explain the purpose, functionality, gameplay dynamics and how circuits are generated (Word document); and describe each quantum gate used in a separate section. Notebook: Task_3_Quantum_TicTacToe/Task3_Quantum_TicTacToe_Completed.ipynb (original kept unchanged beside it)

1.1 What is the purpose of the program?

The program, supplied as Lab 5 (Bahrami, 2026b), is a **teaching game**. Quantum games have long been used to make superposition and measurement intuitive (Goff, 2006). Here every tile of the 3 × 3 board is **one qubit**, and every move adds a **gate** to a nine-qubit circuit. Nobody owns a tile until the end, when the circuit is measured once: a 0 makes the tile X's and a 1 makes it O's. Playing therefore means *designing a quantum circuit*, and the result is probabilistic. The game makes three ideas visible: superposition (undecided tiles), unitary gates as moves, and measurement collapse.

1.2 How is the code organised?

The code has two classes. **`Board`** is the quantum part: it owns the `QuantumCircuit(9, 9)`, adds gates in `make_move`, measures in `measure`, and counts winning lines in `countWinners`. **`Game`** is the classical user interface: it creates buttons with `ipywidgets`, reacts to clicks in `handle_game`, and redraws the board. This separation of quantum logic from the user interface is good software design. It is also how real quantum applications are built: a classical program prepares and sends circuits and then interprets the results.

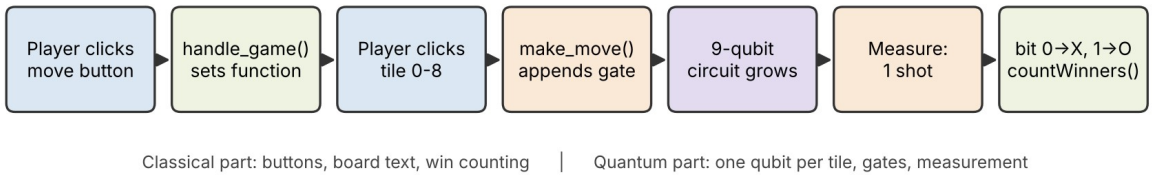


Figure 1. *How one move travels through the code: from a click to a gate, and from measurement to the score*

1.3 What was missing, and what did we complete or fix?

Table 1. *Code completion and fixes (marked [ADDED], [FIXED] or [CHANGED] in the notebook)*

#	Location	Problem in supplied code	Our completion
1	make_move – Not	comment only, no gate	self.qc.x(cell)
2	make_move – O	comment only, no gate	self.qc.ry(+π/4, cell) (toward |1⟩ = O)
3	make_move – X	comment only, no gate	self.qc.ry(−π/4, cell) (toward |0⟩ = X)
4	make_move – SWAP	comment only, no gate	self.qc.swap(cell, self.target)
5	countWinners	winners = () – nobody could ever win	the 8 lines: 3 rows, 3 columns, 2 diagonals
6	SWAP branch	dead inner if; crash if no first tile	guard target ≥ 0; reset target after swap

#	Location	Problem in supplied code	Our completion
7	new()	empty tile '' but __init__ uses ' '	made consistent
8	imports / grid	google.colab widgets only work in Colab	ipywidgets HBox/GridBox (VS Code, Jupyter, Colab)
9	screen updates after a click	callback output only shown in Colab; in Jupyter clicks seemed to do nothing	draw every screen inside an ipywidgets Output widget

An important finding: the supplied comments had the rotation directions swapped. `measure()` maps bit 0 to X and bit 1 to O, but the comments said the O button rotates toward $|0\rangle$ and the X button toward $|1\rangle$. Followed literally, pressing **O** would help player **X**. The game's own rules say "O turns the tile toward a tile owned by O", so we implemented O as a rotation toward $|1\rangle$ and X toward $|0\rangle$ and corrected the comments. Finding a mismatch between comments, code and specification, and resolving it against the specification, is a normal part of investigating someone else's code.

1.4 How does a move become a gate? (Circuit generation)

1. **Start of a round.** `__init__` (and `new()`) adds `reset` and **H** to all nine qubits, then a barrier. Every tile is $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$, i.e. 50% X and 50% O.
2. **Choose a move.** Clicking Not, O, X or SWAP raises a `ValueError` when `int(b.description)` is tried, and the `except` branch stores the move in `board.function`. This is a slightly unusual way to tell text buttons from number buttons.
3. **Choose a tile.** Clicking 0–8 calls `make_move(cell)`, which appends one gate to `self.qc`. For SWAP, the first tile click sets `board.target` and the second click applies the swap.
4. **The circuit grows.** After each move the circuit is redrawn. The text on the board is only a move history; as the original notes say, "the game circuit is the true state of the game".
5. **Measure.** Barrier and nine `measure` instructions are added, and the circuit runs **once** (1 shot) on AerSimulator. Bit i of the result decides tile i, read from the right-hand end of Qiskit's string (`output[9-1-i]`).
6. **Score.** `countWinners` checks the eight lines for each player. Both players can win lines in the same round, because the tiles are decided independently.

1.5 How does a rotation move change the odds?

On the Bloch sphere, $|+\rangle$ sits at angle $\theta = \pi/2$ from $|0\rangle$, and $P(X) = P(0) = \cos^2(\theta/2)$. Each X move applies $R_y(-\pi/4)$, which reduces θ by $\pi/4$ (Figure 2). One X move gives $P(X) = \cos^2(\pi/8) = 0.854$, two moves give 1.000, and **a third move over-rotates** back to 0.854. Moves are rotations, not counters, so "too many" moves undo themselves. This is a quantum effect that classical tic-tac-toe does not have.

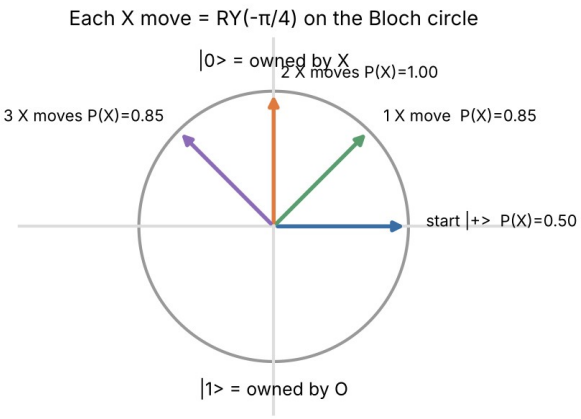


Figure 2. *Effect of repeated X moves on one tile (x–z plane of the Bloch sphere)*

1.6 What happened when we played? (Four recorded games)

Button clicks cannot be saved in a notebook, so we replayed four games as scripts that call exactly the same `make_move` and `measure` code with fixed seeds. For each game we saved the circuit picture, the one-shot result, and a **1000-shot study** of the same circuit, which estimates each tile's chance of becoming X and each player's chance of winning (more X lines than O lines).

Table 2. *Recorded games: moves, generated gates, the one-shot result and 1000-shot win chances*

Game	Moves	Gates added	1-shot lines X:O	X wins /1000	O wins /1000	Draws / 1000
1 Baseline	none	9 H only	4 : 0	392	405	203
2 Diagonals	X on 0, 4, 8; O on 2, 6	+5 RY	4 : 0	705	210	85
3 NOT test	X, X on 4; Not on 4; O on 0; Not on 1	+3 RY, 2 X	0 : 2	135	694	171
4 SWAP	X, X on 0; O, O on 4; SWAP 0 ↔ 8; X × 3 on 2	+7 RY, 1 SWAP	0 : 2	404	289	307

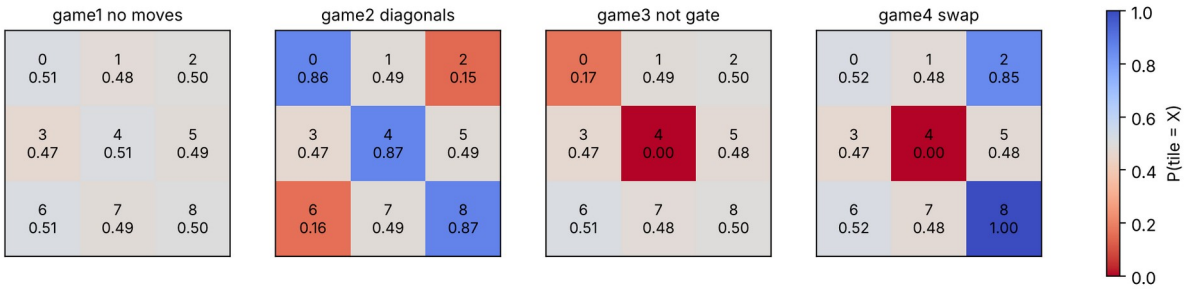


Figure 3. *Chance that each tile becomes X after each recorded game (1000 shots; blue = X likely, red = O likely)*

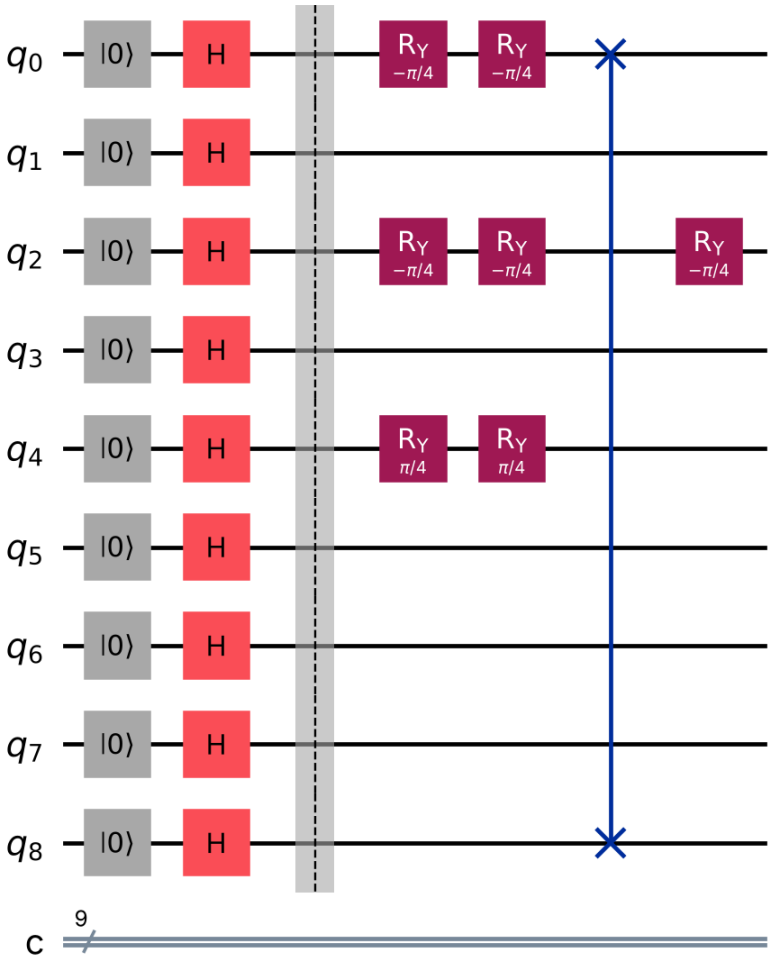


Figure 4. *Circuit generated by Game 4: H start layer, RY moves, and the SWAP between tiles 0 and 8*

1.7 What do the games show? (Analysis)

- **Game 1 (baseline):** with no moves every tile is 50/50, so X and O win about equally often (392 vs 405). The one-shot result (X 4 lines) was simply luck. A single game says little; the 1000-shot study shows the real odds.
- **Game 2:** three X moves on the main diagonal (P(X) ≈ 0.86 each) and two O moves on the other corners raised X's win chance from 39% to 71%. Strategy works, but only in probability.
- **Game 3 (NOT gate):** two X moves made tile 4 certainly X, and NOT then made it certainly O (P(X) = 0.00). NOT on the *undecided* tile 1 did **nothing** (0.49), because $|+\rangle$ is an eigenstate of X: $X|+\rangle = |+\rangle$. This matches the rule "if the tile is not owned, Not does nothing", and it is explained by linear algebra, not by an if-statement.
- **Game 4 (SWAP):** tile 0 was made certainly X and then swapped with tile 8. Afterwards tile 8 is certainly X (1.00) and tile 0 is back to 50/50, because SWAP moved the quantum *states*, not labels. Tile 2 received three X moves and shows the over-rotation (0.85, not 1.00).
- **Circuit growth:** every move adds one gate and does not remove any, so circuit depth rose from 2 to 5. On noisy hardware, longer games would therefore be less reliable.

1.8 Did we also play the game by clicking the buttons?

Yes. To check that the completed game really works for a player, we opened the notebook in **Jupyter Notebook 7** (the environment used in the lectures), ran it, and played three games by clicking the move and tile buttons. Each **Measure** click runs the game circuit once, so every result is a genuine single random outcome.

Table 3. *Three games played interactively in Jupyter Notebook (31 October 2026)*

Game	Buttons clicked	Final board (rows)	Result and what it shows
A	X 4, X 4, O 0, O 8, Not 2	X O O / O X X / O X O	Draw 0–0. Tile 4 (two X moves) was certain X. Tile 0 had one O move (85% O) yet became X: a 15% outcome happened.
B	X 0, X 4, X 8, O 2, O 6	X O O / X X O / O O X	X wins the main diagonal 0–4–8, the three tiles nudged toward X.
C	O 4, O 4, X 1, Not 4, SWAP 0 ↔ 8	X X X / X X O / O O X	X wins 2 lines. Two O moves made tile 4 certain O, then NOT made it certain X, exactly as the theory predicts.

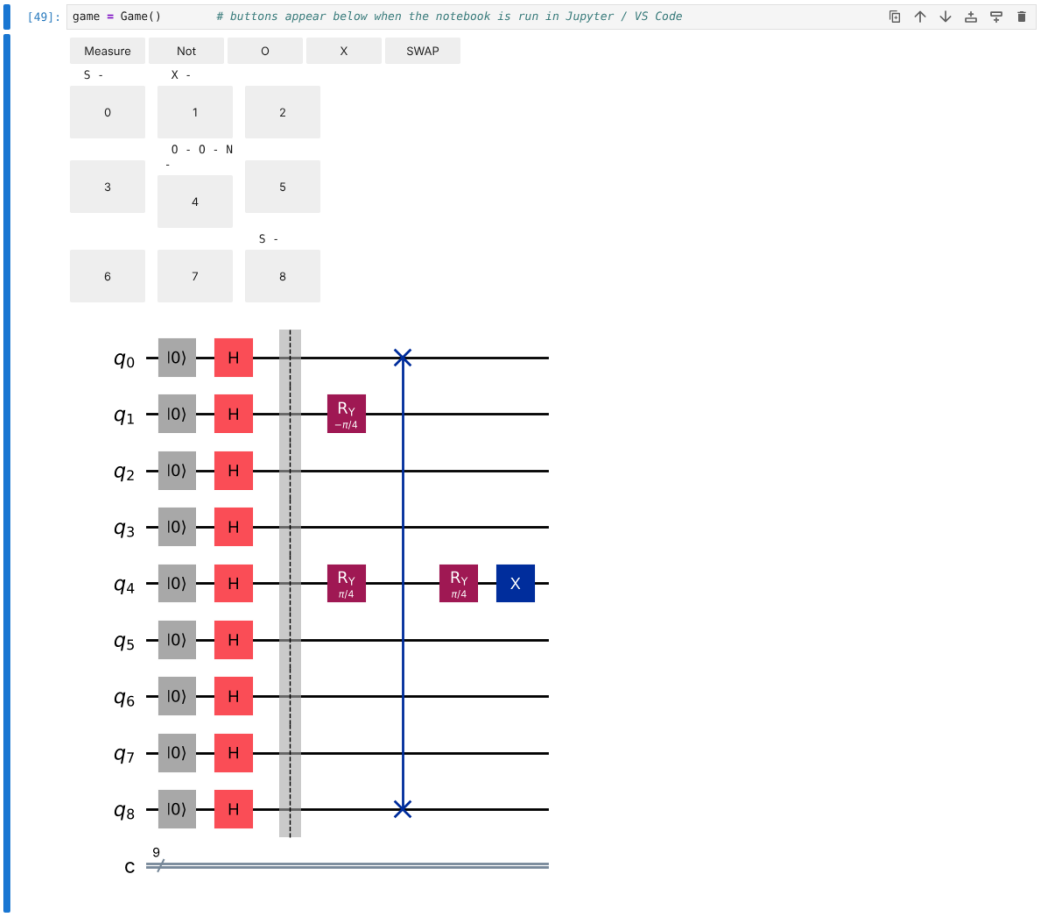


Figure 5. *Game C in Jupyter before Measure: the board shows the move history, and the circuit shows RY, X and the SWAP between tiles 0 and 8*

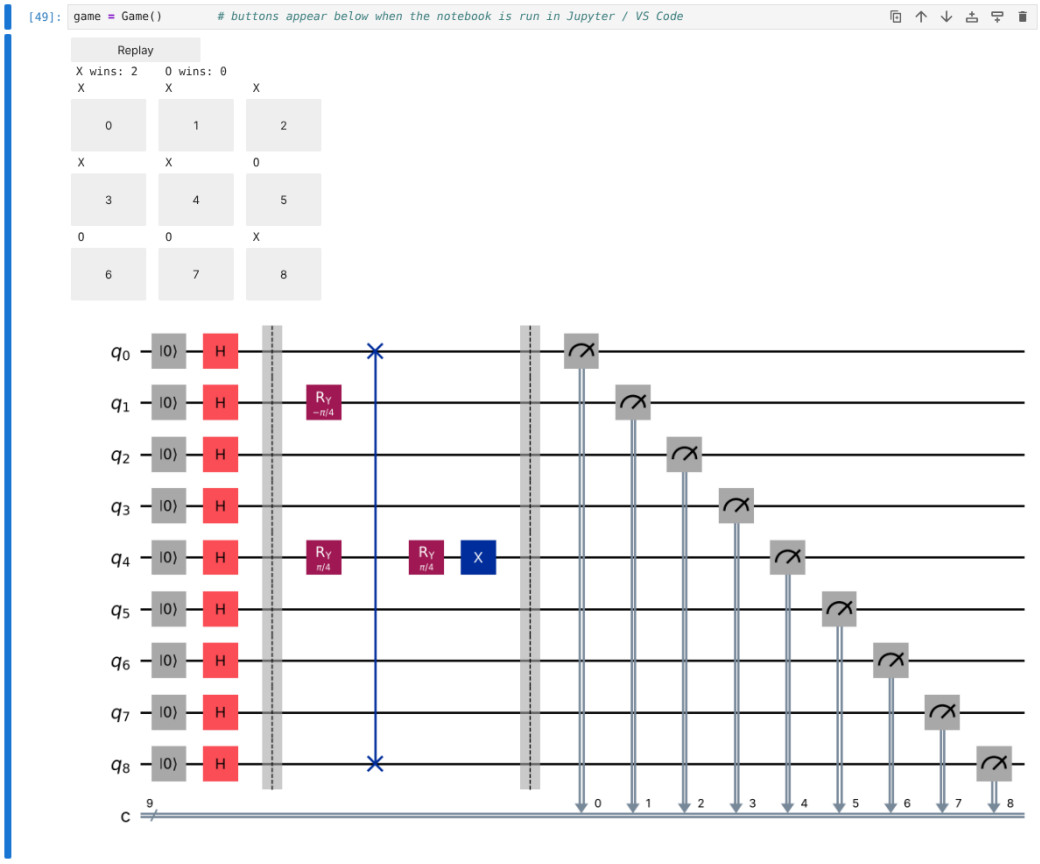


Figure 6. *Game C after Measure: one shot decided every tile; X wins 2 lines*

Playing also exposed a **portability bug**. In the supplied code, every screen update after a click (board, circuit, score) is printed from inside the button's callback. Google Colab shows that output, but Jupyter Notebook and VS Code discard it, so clicks *appeared to do nothing*. We fixed this by drawing all game screens inside an `ipywidgets.Output` widget (marked `[CHANGED]`), after which the game worked in all three environments.

1.9 Can the game circuit run on Quokka?

Yes. The notebook removes the `reset` instructions (qubits already start at $|0\rangle$), translates the Game 2 circuit to H, RY, X and CX gates, exports it to QASM and sends it with 1000 shots. On the Quokka device (quokka2, 4 November 2026) the per-tile chances of X were 0.86, 0.49, 0.14, 0.48, 0.85, 0.50, 0.15, 0.49 and 0.86, the same pattern as Figure 3 and within shot noise of the theory (0.854 for one X move, 0.146 for one O move, 0.5 for untouched tiles). Quokka needed 3.3 ms for the 1000 shots of this nine-qubit circuit. This links the game back to the device used in Tasks 1 and 2.

1.10 Which quantum gates does the game use? (Gate section)

Table 4. *Quantum gates used in circuit generation: characteristics and function in the game*

Gate	Matrix / action	Key characteristic	Function in this game
H (Hadamard)	$(1/\sqrt{2})[[1,1],[1,-1]]$; $\|0\rangle \to \|+\rangle$	creates equal superposition; H·H = I	start layer: every tile undecided (50/50)
$R_y(\theta)$	$[[\cos\theta/2, -\sin\theta/2],[\sin\theta/2, \cos\theta/2]]$	rotation about the y-axis; real amplitudes; changes probabilities smoothly	X move: $R_y(-\pi/4)$ toward $\|0\rangle$; O move: $R_y(+\pi/4)$ toward $\|1\rangle$
X (NOT)	$[[0,1],[1,0]]$; $\|0\rangle \leftrightarrow \|1\rangle$	bit flip; $X\|+\rangle = \|+\rangle$	Not move: flips ownership of a

Gate	Matrix / action	Key characteristic	Function in this game
		(eigenstate)	decided tile
SWAP	exchanges two qubits' states (= 3 CNOTs)	two-qubit, permutes $\|01\rangle \leftrightarrow \|10\rangle$	SWAP move: exchanges two tiles
reset	sets qubit to $\|0\rangle$ (not unitary)	non-reversible initialisation	clean start of each round
measure	projects onto $\|0\rangle$ or $\|1\rangle$	irreversible; Born rule	decides each tile's owner (0 → X, 1 → O)
barrier	no operation	visual / compiler boundary	separates start, moves and measurement in the drawing

1.11 How do these gates fit the wider theory of quantum logic gates?

Every quantum gate is a **unitary matrix** ($U^\dagger U = I$), so every gate is **reversible**: applying $U^\dagger$ undoes it. `reset` and `measure` in the table above are the exceptions; they are not gates in this strict sense because they cannot be undone, and measurement follows the Born rule (Nielsen & Chuang, 2010; "Quantum logic gate," n.d.). Gates in sequence combine by **matrix multiplication**, written in the reverse of the drawing order (X then Y gives the matrix YX), and gates on different qubits combine by the **tensor product**. That is how the nine-qubit game state is built from nine single-qubit operations.

A gate set is **universal** if it can approximate any unitary operation; examples are {H, T, CNOT}, the Clifford+T set, or arbitrary rotations plus CNOT (Barenco et al., 1995; "Quantum logic gate," n.d.). This idea explains two of our findings:

- **The game's gates are not universal and cannot entangle tiles.** H, R_y and X keep every amplitude real, and SWAP only moves states between tiles. The game has no entangling gate such as CNOT, so its nine tiles always stay independent. Adding a CNOT move ("link two tiles") would make the gate set universal and allow entangled tiles.
- **Quokka's gate set is universal.** Its QASM gates include Rx, Ry, Rz and CNOT, which form a universal set. That is why `to_quokka_gates()` can always rewrite S, T or the Toffoli into supported gates (Task 2B). The rewritten circuit can differ only by a **global phase**, which never changes a measurement result.

1.12 What are the strengths and limits of the program?

Strengths: the program is short, has clear quantum/classical separation and a good visual circuit, and it demonstrates superposition, rotation and measurement in a playful way. **Limitations:** (1) tiles are never *entangled*, because the only two-qubit gate is SWAP, which keeps tiles independent; adding a CNOT move would create linked tiles and is a natural improvement. (2) One-shot measurement makes single games very random. (3) The board text is "not 100% accurate", so it can mislead players. (4) Using exceptions to tell buttons apart is fragile. (5) There is no turn order, so one player can make every move. Despite these limits, as a teaching tool it shows clearly that a quantum program is a circuit that is built step by step and only gives answers when measured.

References

Bahrami, H. (2026b). *Lab 5 — Quantum tic-tac-toe* [Unpublished Jupyter notebook, Quantum_Tic_Tac_Toe__AS2.ipynb]. MSE802 Quantum Computing, Yoobee College of Creative Innovation.

Barenco, A., Bennett, C. H., Cleve, R., DiVincenzo, D. P., Margolus, N., Shor, P., Sleator, T., Smolin, J. A., & Weinfurter, H. (1995). Elementary gates for quantum computation. *Physical Review A*, *52*(5), 3457–3467. https://doi.org/10.1103/PhysRevA.52.3457

Goff, A. (2006). Quantum tic-tac-toe: A teaching metaphor for superposition in quantum mechanics. *American Journal of Physics*, *74*(11), 962–973. https://doi.org/10.1119/1.2213635

Nielsen, M. A., & Chuang, I. L. (2010). *Quantum computation and quantum information* (10th anniversary ed.). Cambridge University Press. https://doi.org/10.1017/CBO9780511976667

Quantum logic gate. (n.d.). In *Wikipedia*. Retrieved October 31, 2026, from https://en.wikipedia.org/wiki/Quantum_logic_gate